CI/CD Pipelines

How Applications Go Live & Project Environments

A Complete Guide for Data Engineering

Submitted By

Vaishali S

Program: Data Engineering Training

Date: 12 - 05 - 2026

SECTION 1 — What are CI/CD Pipelines?

Before CI/CD existed, developers would write code for weeks or months, then try to merge everything together at once. This caused what is known as 'integration hell' — a nightmare of conflicts, broken code and failed releases.

CI/CD — Continuous Integration / Continuous Delivery (or Deployment) — is an automated pipeline that takes code from a developer's laptop all the way to production without manual steps.

Think of CI/CD like an airport conveyor belt — you place your bag (code) at one end, and it goes through scanning (testing), tagging (build), and loading (deployment) automatically before reaching the plane (production).

CI — Continuous Integration

Continuous Integration means every developer pushes code to a shared repository frequently (multiple times a day). Each push automatically triggers a build and runs all tests. If anything breaks, the team is notified immediately.

CI Step	What Happens
Code Push	Developer pushes code to Git (GitHub / GitLab / Bitbucket)
Trigger	CI server detects the push and starts the pipeline automatically
Build	Code is compiled or packaged (e.g. Maven build, Docker image built)
Unit Tests	All unit tests run automatically — pass or fail in minutes
Code Analysis	SonarQube or similar scans for code quality, bugs, vulnerabilities
Notify	Team gets Slack/email alert — green (pass) or red (fail)

CD — Continuous Delivery vs Continuous Deployment

These two are often confused. Here's the exact difference:

	Continuous Delivery	Continuous Deployment
Definition	Code is always ready to deploy — needs 1 manual approval click	Code goes live fully automatically with zero human click
Human step?	YES — someone approves production release	NO — fully automated end to end
Best for	Banks, healthcare — need sign-off before going live	SaaS products — Netflix, Amazon — ship 100s of times daily
Risk level	Lower — human checks before production	Needs strong test coverage to be safe

The Full CI/CD Pipeline — Step by Step

Here is the complete journey of code from a developer's machine to production:

#	Stage	What Happens	Tools Used
1	**Source Control**	Developer writes code and pushes to Git branch	Git, GitHub, GitLab, Bitbucket
2	**Build**	Code compiled, dependencies resolved, artifact created	Maven, Gradle, npm, Docker
3	**Unit Testing**	Automated tests run to check individual code units	JUnit, PyTest, Mocha, Jest
4	**Code Quality**	Static code analysis for bugs, smells, vulnerabilities	SonarQube, ESLint, Checkstyle
5	**Integration Test**	Test how different services/modules work together	Postman, Selenium, REST Assured
6	**Staging Deploy**	Deploy to staging environment — exact copy of production	Kubernetes, Ansible, Terraform
7	**UAT / QA Test**	Business team tests on staging for final sign-off	Manual + automated test suites
8	**Approval Gate**	Team lead / manager approves production release	Jenkins, GitHub Actions approval step
9	**Production Deploy**	Code goes LIVE — real users can now use the feature	AWS, Azure, GCP, Heroku, Docker
10	**Monitor**	Watch app performance, errors, logs in production	Grafana, Prometheus, Datadog, CloudWatch

Popular CI/CD Tools

Tool	What It Does	Used By
Jenkins	Open-source CI/CD automation server	Most enterprise companies
GitHub Actions	CI/CD built directly into GitHub	Startups, open source projects
GitLab CI/CD	Integrated pipeline inside GitLab	Mid-size to large companies
CircleCI	Fast cloud-based CI/CD service	Tech startups
Azure DevOps	Microsoft's full CI/CD platform	Microsoft ecosystem companies
AWS CodePipeline	CI/CD native to AWS cloud	AWS-hosted applications

SECTION 2 — How Does an Application / Product Go Live?

Going Live (also called a Release or Launch) is the process of making your application available to real end users in production. It is not a single action — it is a carefully planned series of steps involving development, testing, approvals and deployment strategies.

Phase 1 — Development Complete

- All features for the release are coded and code-reviewed
- Pull requests merged into the main/release branch
- CI pipeline runs automatically — build passes, all unit tests green
- Code quality gates passed (no critical bugs flagged by SonarQube)

Phase 2 — Testing Phase

- Code deployed to DEV environment → developers do smoke testing
- Code promoted to QA environment → QA team runs full regression tests
- Performance and load testing done (can the app handle 10,000 users?)
- Security scanning — check for vulnerabilities before going live
- Bug fixes done and re-tested — iterate until stable

Phase 3 — UAT (User Acceptance Testing)

- Code deployed to UAT/Staging — exact mirror of production
- Business stakeholders and real users test actual workflows
- Product Owner reviews and accepts or rejects completed features
- Sign-off obtained from business team before production release

Phase 4 — Release Planning

- Release date and time window decided (usually off-peak hours — 2AM to 5AM)
- Rollback plan prepared — if something breaks, how do we undo it?
- Go-live checklist prepared and signed off by team leads
- Database migration scripts reviewed and tested separately
- All dependent teams notified — support team, ops team, client

Phase 5 — Production Deployment

This is the actual Go-Live moment. Different deployment strategies are used depending on the risk level:

Strategy	How It Works	When to Use
Big Bang	All users switched to new version at once	Small apps, low risk
Blue-Green	Two identical environments — traffic switches from Blue (old) to Green (new) instantly	Zero downtime needed
Canary Release	New version released to 5% of users first — watch for issues — then roll to 100%	High-traffic apps
Rolling Deploy	Servers updated one by one — app stays available throughout	Containerized / cloud apps

Feature Flags	Code deployed but feature switched OFF — turned on remotely per user group	Safe gradual launches

Phase 6 — Post Go-Live Monitoring

- Monitor dashboards for errors, latency, CPU/memory spikes
- Support team on standby for first 24-48 hours after go-live
- Compare metrics before vs after release
- If critical bug found — rollback to previous version immediately
- Post-release retrospective — what went well, what to improve next time

SECTION 3 — Project Environments Explained

Every software project uses multiple environments — separate isolated spaces where code runs at different stages. Each environment has a specific purpose, and code flows through them in order before reaching real users.

Think of environments like stages in a film production — Script (Dev) → Rehearsal (QA) → Dress Rehearsal (Staging/UAT) → Live Show (Production). Each stage has a specific role and audience.

Environment	Purpose	Who Uses It	Data Used	Stable?
LOCAL	Developer writes and tests code on own machine	Individual developer only	Mock / dummy data	No — changes constantly
DEV	Team integration — all developers' code merged here first	Dev team	Test data	Somewhat
QA / TEST	QA team runs all test cases — functional, regression, API	QA engineers	Sanitised test data	Yes — controlled
STAGING / UAT	Business testing — exact copy of production for final sign-off	Business analysts, Product Owner, clients	Near-real / masked data	Very stable
PRODUCTION	LIVE — real users, real data, real money	End customers / all users	Real production data	Must be 100% stable

LOCAL Environment

Local is the developer's own laptop or desktop. Code is written, run and tested here before being pushed to any shared environment.

- Each developer has their own local setup — no one else is affected by changes

- Uses mock data, local databases, and stub APIs to simulate real services
- Developer runs the app on localhost (e.g. http://localhost:3000)
- No deployment process — just run the code directly on the machine
- Changes are pushed to DEV only when the developer is satisfied

DEV (Development) Environment

The DEV environment is the first shared server environment. All developers push their code here. It is the earliest integration point where everyone's code runs together.

- Automatically deployed via CI pipeline on every code push to main branch
- Used for developer-level testing — basic smoke tests and feature checks
- Often unstable — developers are constantly pushing changes
- Multiple features being built simultaneously may conflict here
- Not shown to clients or business teams — internal dev use only
- Typical URL: dev.amazon-app.com or internal IP address

QA / TEST Environment

The QA environment is dedicated to the Quality Assurance team. Once DEV is stable, code is promoted here for thorough testing before business teams see it.

- QA runs functional tests — does every feature work as specified?
- Regression testing — does the new code break any old features?
- API testing — are all endpoints returning correct responses?
- Performance testing — can the system handle expected load?
- Bug reports raised here → fixed in DEV → re-deployed to QA → re-tested
- Uses sanitised copies of real data — no real customer information
- Typical URL: qa.amazon-app.com or test.amazon-app.com

STAGING / UAT Environment

Staging (also called UAT — User Acceptance Testing) is the most important pre-production environment. It is an exact replica of production — same server specs, same database structure, same configuration.

- Business stakeholders test real workflows — 'does this do what I asked for?'
- Product Owner accepts or rejects features based on acceptance criteria
- Client demos happen here — the client sees the product before go-live
- Final performance and security testing done in staging
- Database migration scripts tested here before running on production
- Once UAT sign-off is received, code is approved for production release
- Typical URL: staging.amazon-app.com or uat.amazon-app.com

PRODUCTION Environment

Production is the LIVE environment. This is where real users interact with your application. It uses real data, real servers, real money. Any bug here affects actual customers immediately.

- Highest security, highest availability, highest stability requirements
- Access is restricted — only senior engineers and DevOps can deploy here
- All changes go through full CI/CD pipeline and approvals before reaching here
- Monitored 24/7 — any downtime costs the business money and reputation
- Never tested with new features directly — always staging first
- Rollback plan always ready — if go-live breaks anything, revert within minutes
- Typical URL: amazon.in or app.amazon.com — the real website!

How Code Flows Across Environments

The journey of code from developer to production always follows this order:

LOCAL	→	DEV	→	QA	→	STAGING	→ PROD
Write code		Merge & build		Test all cases		UAT sign-off	GO LIVE!

Environment Comparison — Data Engineering Context

In Data Engineering projects, environments apply to data pipelines, ETL workflows and databases too — not just web apps:

Environment	Data Pipeline	Database	Airflow DAGs
LOCAL	Run DAG locally with sample CSV	Local SQLite / Docker Postgres	Run manually on local Airflow
DEV	Test pipeline with dev DB connection	Dev database — dummy records	Deployed to dev Airflow server
QA	Run full pipeline with test dataset	QA DB — sanitised real data copy	Scheduled runs tested by QA team
STAGING	Full pipeline with prod-like volume	Masked production data snapshot	Full schedule tested — stakeholder review
PRODUCTION	Live pipeline processing real data	Real production database — live data	Live DAGs running on schedule 24/7

Summary

CI/CD, Go-Live processes and Environments work together as a complete system that ensures software is built, tested and deployed reliably and safely. Here is a one-line recap of everything:

Concept	One Line Summary
CI — Continuous Integration	Every code push automatically builds and tests — catch bugs same day
CD — Continuous Delivery	Code always ready to ship — 1 manual approval before production
CD — Continuous Deployment	Fully automated — code goes live with zero human click
CI/CD Pipeline	Automated conveyor belt: Code → Build → Test → Deploy → Monitor
Go-Live	Planned release process: Dev → QA → UAT → Approval → Production
LOCAL	Developer's own machine — write and first test code here
DEV Environment	First shared server — all developers push code here for integration
QA Environment	QA team tests all features — bug reports raised and fixed here
STAGING / UAT	Exact copy of production — business team gives final sign-off
PRODUCTION	LIVE — real users, real data, monitored 24/7